

TECHNICAL INTERVIEW · JUNE 2026

Interview **Prep** Guide

Principia Mentis GmbH · Senior Backend Web Developer

-
- Candidate: Vinitha Pukazh Bagya R
 - Format: ~2-hour technical interview via Microsoft Teams
 - Interviewers: Nikolaus Kirstein & Andreas Bergmaier
 - Topics: SQL Schema · JavaScript/TypeScript · Node.js · Web Technologies



What to Expect



The interview has **two parts**: (1) a deep-dive into your submitted `schema.sql` challenge, then (2) broader technical questions on JS/TS, Node.js, SQL, and web technologies. Duration is approximately 2 hours.



Their philosophy: "We're mostly interested in your thought process, the questions you encounter along the way, the assumptions, decisions and trade-offs you make." — Think out loud. Acknowledge trade-offs. Don't try to give a "perfect" answer.



AI transparency: They specifically asked you to be open if you used AI. Prepare a brief, honest answer about how you used it and what you contributed yourself.

PART 1 — Schema Challenge Deep-Dive

They will walk through your schema.sql line by line. Know every design decision cold.

A Tree Structure Design (Question 1)

1 "Walk us through how you modelled the tree structure."

I used a **hybrid approach**: an adjacency list (parent_id) plus a materialized path (path TEXT column).

- **Adjacency list** (parent_id FK → task_node.id) gives a simple, natural parent-child link and easy writes.
- **Materialized path** stores each node's full ancestry as a delimited string, e.g. /1/4/7/. This enables fast ancestor/descendant queries with a simple LIKE '/1/4/%' — no recursive CTE needed at query time.

I chose this hybrid because tree reads (permission checks, subtree traversals) are far more frequent than writes in a project management tool.

2 "What does the path column look like in practice?"

Each node stores the IDs of all its ancestors plus its own ID, delimited by slashes. For example:

- Root node (id=1): /1/
- Child of root (id=4): /1/4/
- Grandchild (id=7): /1/4/7/

To get all descendants of node 4: WHERE path LIKE '/1/4/%'

To get all ancestors of node 7: WHERE '/1/4/7/' LIKE path || '%'

3 "Compare tree storage strategies — why not nested sets or a closure table?"

Approach	Read Speed	Write Speed	Query Style
Adjacency List	Slow (recursive)	Fast	WITH RECURSIVE CTE
Materialized Path ✓	Fast	Medium	Simple LIKE query
Nested Sets (left/right)	Fast	Very slow	Range query
Closure Table	Fast	Medium	Extra join table

My hybrid gets the best of adjacency list (simple structure, foreign key integrity) and materialized path (fast read queries) without the write-heavy complexity of nested sets.

4 "What happens to the path when a node is moved to a different parent?"

Moving a node requires updating path for the node itself and all its descendants. I would do a cascading UPDATE:

SQL — MOVE NODE SUBTREE

```
UPDATE task_node SET path = new_parent_path || substring(path FROM length(old_path) + 1) WHERE  
path LIKE old_path || '%';
```

This is a known trade-off of materialized paths — moves are more expensive than with a pure adjacency list. For a project management tool where moves are rare, this is acceptable.

5

"Write a recursive SQL query on this schema — e.g. get all descendants of a node."

SQL – RECURSIVE CTE: ALL DESCENDANTS

```
WITH RECURSIVE subtree AS ( -- Anchor: starting node SELECT id, parent_id, text, 1 AS depth FROM
task_node WHERE id = :start_node_id UNION ALL -- Recursive: join children of current set SELECT
tn.id, tn.parent_id, tn.text, st.depth + 1 FROM task_node tn JOIN subtree st ON tn.parent_id =
st.id ) SELECT * FROM subtree ORDER BY depth;
```

To get ancestors instead: reverse the join condition — JOIN subtree st ON st.parent_id = tn.id — starting from the target node upward.

6

"What are the risks of recursive CTEs on deep trees?"

- **Infinite loops:** If there's a cycle in parent_id (data corruption), the CTE loops forever. Mitigate with a depth < 100 guard or PostgreSQL's built-in cycle detection.
- **Performance:** Deep trees with many nodes can be slow — each recursive step is a join. For very deep trees, the materialized path approach is better for reads.
- **Stack depth:** Very deep recursion can hit server limits.

This is why I chose the materialized path approach for permission queries — it avoids recursion entirely at query time.

c Roles & Permissions Schema (Question 3 — Core)

7 "Walk us through your roles and permissions design."

My schema has four tables working together:

- **user_role:** Assigns a user a role at a specific node — (user_id, node_id, role_id). One user can have different roles at different nodes.
 - **role_permission:** Defines what permissions a role grants, and crucially, *in which scope* — using a node_scope ENUM: node_only, descendants, or ancestors.
 - At query time, I resolve which scope applies by comparing the target node's path to the assignment node's path.
- This lets me express rules like: "Viewer gets ViewStructure on the assigned node and its descendants, but only ViewStructure (not ViewData) on ancestors."

8 "Why a node_scope ENUM rather than separate boolean columns or separate join tables?"

Three booleans (at_node, at_descendants, at_ancestors) would work too — this is a valid trade-off. I chose an ENUM because:

- It's readable and self-documenting — the scope is explicit
- The primary key (role_id, permission_id, node_scope) is clean and prevents duplicates
- It maps directly to the three relationship types in the problem statement

The downside is that you can't express partial scope (e.g., "only direct children"). If that requirement emerged, I'd switch to a more flexible model.

9 "Show the role-permission matrix for all four roles."

Role → Scope	ViewStructure	EditStructure	ViewData	EditData
Admin @ node_only	✓	✓	✓	✓
Admin @ descendants	✓	✓	✓	✓
Admin @ ancestors	✓	✓	✓	✓
Requester @ node_only	✓	✓	✓	✓
Requester @ descendants	✓	✓	✓	✓
Requester @ ancestors	✓	—	✓	—
Responsible @ node_only	✓	✓	✓	✓
Responsible @ descendants	✓	✓	✓	✓
Responsible @ ancestors	✓	—	—	—
Viewer @ node_only	✓	—	✓	—
Viewer @ descendants	✓	—	✓	—
Viewer @ ancestors	✓	—	—	—

10 "Explain your big permission-resolution query step by step."

The query finds all effective permissions for a user on a specific target node:

- **Step 1:** Start with the target node.
- **Step 2:** JOIN all nodes gn ("granted from node") that have a relationship to target — either same node, ancestor of target, or descendant of target. This uses LIKE on the path column.
- **Step 3:** Filter user_role — only where the given user has a role assignment at node gn.
- **Step 4:** Match role_permission using a CASE expression to determine which scope applies: if gn = target → node_only; if gn is ancestor of target → ancestors; if gn is descendant → descendants.
- **Step 5:** Return the permission name, the role that granted it, and the node it was granted from.

This resolves permissions dynamically at query time without pre-computing or caching.

D Known Weaknesses & Probing Questions

⚠ They will probe for weaknesses in your design. Having good answers to these shows maturity and honest engineering thinking.

11 "Your path is a plain TEXT column — how do you enforce consistency?"

Honestly, a plain TEXT column has no DB-level format constraint. In production I'd add:

- A CHECK constraint or trigger to validate the format matches `/(\d+)/+`
- Application-level validation on insert/update
- Alternatively, use PostgreSQL's native **ltree extension** which is purpose-built for this — it handles path indexing, format validation, and provides built-in operators like `@>` (ancestor of) and `<@` (descendant of)

I kept it as plain TEXT to avoid extension dependency, but ltree would be the production-grade choice.

12 "Can a user get duplicate permissions from your query?"

Yes — if a user has role assignments at multiple nodes in the tree, the same permission can appear multiple times from different sources. The query as written returns all grants with their origin, which is useful for auditing. To get a deduplicated list of effective permissions, wrap it in a `SELECT DISTINCT p.name` or use a subquery.

13 "What about performance at scale — large trees, many users?"

- **Indexing path:** LIKE `'/1/4/%'` queries benefit from a B-tree index on path (prefix queries work). For PostgreSQL, a GIN index or the `ltree` extension with GiST indexing is even better.
- **Caching permissions:** For hot paths, compute and cache a user's effective permissions (e.g. in Redis) and invalidate on role changes.
- **Denormalisation:** A closure table (all ancestor-descendant pairs pre-stored) makes permission queries a simple join — at the cost of more storage and write overhead.

14 "How would you handle conflicting permissions — most permissive or most restrictive wins?"

My current schema uses a **most-permissive / union model** — if any role grant gives a permission, the user has it. To implement most-restrictive wins or explicit deny, I'd need to add a `grant_type` column (allow / deny) to `role_permission`, then resolve with priority logic in the query. This is a deliberate design choice worth discussing with the team based on their actual product requirements.

PART 2 — Technical Questions

JavaScript/TypeScript · Node.js · SQL · Web Technologies

E JavaScript & TypeScript

15 "What is the event loop in JavaScript?"

JavaScript is single-threaded. The event loop coordinates three layers:

- **Call stack:** Executes synchronous code.
- **Microtask queue:** Holds resolved Promises and `queueMicrotask()` callbacks. Drained completely before each macrotask.
- **Macrotask queue:** Holds `setTimeout`, `setInterval`, I/O callbacks.

After each call stack frame, the engine drains all microtasks, then picks one macrotask, then drains microtasks again — repeat.

16 "Explain callbacks → Promises → async/await evolution."

- **Callbacks:** Pass a function to be called when async work completes. Leads to "callback hell" — deeply nested, hard to read/handle errors.
- **Promises:** Object representing a future value. Chainable with `.then()`/`.catch()`. Much more readable, error propagation works naturally.
- **async/await:** Syntactic sugar over Promises. Makes async code look synchronous. Use `try/catch` for error handling. Under the hood it's still Promises.

In my Node.js work at Guidewire and Thoughtworks, I exclusively used `async/await` with proper error boundaries.

17 "interface vs type in TypeScript — when do you use each?"

- **interface:** For defining the shape of objects. Supports declaration merging (multiple declarations with same name merge). Preferred for public APIs and class contracts.
- **type:** More flexible — supports unions (`string | number`), intersections (`A & B`), mapped types, conditional types. Cannot be merged.

Rule of thumb: use `interface` for object shapes you'll extend; use `type` for unions, utility types, or complex compositions.

18 "What is a closure and give a practical example?"

A closure is a function that retains access to variables from its outer scope even after the outer function has returned.

```
function makeCounter() { let count = 0; // enclosed variable return () => ++count; } const counter = makeCounter(); counter(); // 1 counter(); // 2 — count persists across calls
```

Closures are fundamental to patterns like memoization, partial application, and module encapsulation.

19 "What are TypeScript generics and when should you use them?"

Generics allow writing reusable, type-safe code where the concrete type is provided at call site.

```
function identity<T>(arg: T): T { return arg; } const s = identity<string>("hello"); // s: string
type ApiResponse<T> = { data: T; error: string | null };
```

Use generics when the same logic applies to multiple types and you want to preserve type information (as opposed to using `any`).

"What is the difference between == and === in JavaScript?"

== performs type coercion before comparison ("1" == 1 is true). === is strict equality — no coercion, checks both value and type. Always use === unless you explicitly need coercion. This is a common source of bugs in JavaScript.

21 "How does Node.js achieve concurrency with a single thread?"

Node.js uses **non-blocking I/O** via **libuv**. When an I/O operation (DB query, file read, HTTP call) is initiated, Node registers a callback and continues executing other code. libuv uses an OS-level event notification system (epoll/kqueue) and a thread pool for operations that can't be made async natively (e.g. file system on some OSes).

This means one Node.js process can handle thousands of concurrent connections — as long as the main thread isn't blocked by CPU-intensive work. For CPU-heavy tasks, use Worker Threads or delegate to a separate service.

22 "Difference between `process.nextTick()`, `setImmediate()`, and `setTimeout(fn, 0)`?"

- **`process.nextTick()`**: Runs before the next I/O event — in the current iteration of the event loop, after the current operation completes. Highest priority.
- **`setImmediate()`**: Runs in the "check" phase, after I/O callbacks in the same loop iteration.
- **`setTimeout(fn, 0)`**: Runs in the "timers" phase of the next loop iteration. Slightly later than `setImmediate` in I/O context.

Order: `nextTick` → microtasks → I/O → `setImmediate` → `setTimeout`

23 "What is middleware in Express.js?"

Middleware are functions with the signature `(req, res, next)` that sit in the request-response pipeline. They can read/modify `req` and `res`, run async operations, and either end the response or call `next()` to pass to the next middleware.

- Common uses: authentication, logging, error handling, body parsing, CORS headers
- Error-handling middleware has 4 params: `(err, req, res, next)`

I've used this pattern extensively — e.g. auth middleware that verified JWTs before reaching route handlers.

24 "How do you manage DB connection pooling in Node.js with PostgreSQL?"

Using the `pg` (node-postgres) library with a Pool:

```
import { Pool } from 'pg'; const pool = new Pool({ connectionString: process.env.DATABASE_URL,
max: 20, // max pool size idleTimeoutMillis: 30000, connectionTimeoutMillis: 2000 });
```

Pool reuses connections instead of opening a new TCP connection per query. Key settings: `max` (pool size matching DB `max_connections`), `idleTimeoutMillis`, and `connectionTimeoutMillis`. Always release connections back to the pool (use `pool.query()` which auto-releases, or manually with `client.release()`).

25 "How do you handle errors globally in a Node.js application?"

Three layers of error handling:

- **Application level**: `try/catch` in async functions; error-handling Express middleware
- **Promise rejections**: `process.on('unhandledRejection', (err) => { ... })`
- **Synchronous exceptions**: `process.on('uncaughtException', (err) => { ... })` — log and gracefully shut down

Best practice: never swallow errors silently; always log with enough context; use structured logging (e.g. Winston, Pino) with correlation IDs.

26 "What is a database index and when should you add one?"

An index is a separate data structure (typically a B-tree) that allows the DB to find rows matching a condition without scanning the entire table.

- **Add indexes on:** columns used in WHERE, JOIN ON, ORDER BY, GROUP BY
- **Avoid over-indexing:** Each index slows down INSERT/UPDATE/DELETE and uses storage
- **Composite indexes:** Column order matters — (a, b) helps queries on a or (a, b) but not b alone
- **Partial indexes:** Index only rows matching a condition — e.g. WHERE deleted_at IS NULL

27 "What are window functions and when do you use them?"

Window functions perform calculations across a set of rows related to the current row, without collapsing them into a single output row.

```
-- Rank employees by salary within each department
SELECT name, department, salary, RANK() OVER
(PARTITION BY department ORDER BY salary DESC) AS rank FROM employees;
```

Common functions: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER(), AVG() OVER(). Use them for running totals, ranking, comparing current row to previous/next.

28 "CTE vs subquery — what's the difference and which is faster?"

- **CTE (WITH clause):** Named, readable, can be referenced multiple times in the query, required for recursion. In PostgreSQL, non-recursive CTEs are sometimes optimisation fences (treated as materialised).
- **Subquery:** Inline, anonymous. The optimiser can sometimes push predicates into subqueries more aggressively.
- **Performance:** In modern PostgreSQL (12+), CTEs are generally inlined and optimised like subqueries unless marked MATERIALIZED. Use CTEs for readability; profile with EXPLAIN if performance matters.

29 "How do you diagnose a slow query in PostgreSQL?"

```
EXPLAIN ANALYZE SELECT * FROM task_node WHERE path LIKE '/1/4/%';
```

Look for:

- **Seq Scan** on large tables → missing index
- **High actual rows vs estimated rows** → stale statistics, run ANALYZE
- **Nested Loop on large sets** → may need a Hash Join, or the join condition is unindexed
- **High cost nodes** → start optimisation there

Also check pg_stat_statements for most-called / slowest queries across the whole system.

30 "What is a transaction and what are isolation levels?"

A transaction groups SQL statements into an atomic unit — either all succeed or all roll back. Isolation levels control how concurrent transactions see each other's changes:

- **READ UNCOMMITTED:** Can see uncommitted changes (dirty reads) — rarely used in PostgreSQL
- **READ COMMITTED (default):** Only sees committed data. Each statement gets a fresh snapshot.
- **REPEATABLE READ:** Snapshot taken at transaction start — consistent reads throughout
- **SERIALIZABLE:** Full isolation — behaves as if transactions ran serially

Higher isolation = fewer anomalies but more locking/contention. Choose based on your consistency requirements.

31 "REST vs GraphQL — when would you pick one over the other?"

- **REST:** Resource-based, uses HTTP verbs (GET/POST/PUT/DELETE), simple to cache, widely understood. Great for simple CRUD APIs and public APIs.
- **GraphQL:** Client specifies exactly what data it needs — avoids over-fetching and under-fetching. Excellent for complex, nested data (like a task tree!) and multiple client types (mobile/web with different needs).

For this project management tool with a tree structure, GraphQL would be interesting — clients could query exactly the nodes and fields they need. I'd lean REST for simplicity unless the UI has varied data needs.

32 "What is CORS and how do you handle it in a Node.js API?"

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks web pages from making requests to a different origin (domain/port/protocol) than the page was served from.

```
import cors from 'cors'; app.use(cors({ origin: ['https://app.principiamentis.com'], methods: ['GET', 'POST', 'PUT', 'DELETE'], credentials: true }));
```

The server adds Access-Control-Allow-Origin headers. Preflight OPTIONS requests are sent by browsers for non-simple requests.

33 "How does JWT authentication work?"

A JWT (JSON Web Token) has three base64-encoded parts: `header.payload.signature`.

- **Header:** Algorithm (e.g. HS256) and token type
- **Payload:** Claims — user ID, roles, expiry (not encrypted — don't store secrets here!)
- **Signature:** HMAC of header + payload using a secret key — server verifies this to trust the token

Flow: user logs in → server signs a JWT → client stores it → client sends it in `Authorization: Bearer <token>` header → server validates signature without hitting the DB. Stateless.

Downside: JWTs can't be invalidated before expiry without a token blacklist. Use short expiry + refresh tokens.

34 "What is event-driven architecture and where have you used it?"

In event-driven architecture, services communicate by publishing and subscribing to events (messages) via a message broker, rather than calling each other directly (synchronous HTTP).

My experience: At Thoughtworks, I built a Workday integration that published user change events to GCP Pub/Sub. Downstream services subscribed and updated their own data stores asynchronously. This decoupled the services, improved resilience (if a subscriber is down, events queue up), and saved ~30 minutes per change request.

Benefits: loose coupling, scalability, resilience. Trade-offs: eventual consistency, harder to trace flows, need idempotency.

Directly relevant to this job! My approach:

- **Auth middleware:** Validate JWT, attach user object with roles to `req.user`
- **Permission middleware:** For each route, check if user's role has the required permission — query the DB or check a cached permission set
- **Attribute-based:** For complex cases (like this tree), resolve permissions at the resource level — pass the node ID to a permission service that runs the schema query

```
// Middleware example
const requirePermission = (permission: string) => async (req, res, next) =>
{
  const nodeId = req.params.nodeId;
  const hasPermission = await checkPermission(req.user.id, nodeId, permission);
  if (!hasPermission) return res.status(403).json({ error: 'Forbidden' });
  next();
};
```

HTTP caching reduces server load and latency by storing responses:

- **Cache-Control:** Directives like `max-age=3600`, `no-cache`, `private`, `public`
- **ETag:** Server sends a hash of the resource; client sends it back in `If-None-Match`; server returns 304 Not Modified if unchanged
- **Last-Modified / If-Modified-Since:** Similar but time-based

For a permission-heavy API, be careful caching responses — cached data may reflect stale permissions. Use `no-store` or short `max-age` for permission-sensitive endpoints.

Day-of Checklist & Questions to Ask Them

Final preparation steps and how to leave a great impression



Before the Interview

- ☐ Re-read your schema.sql line by line — know every line cold and be ready to explain it
- ☐ Set up a live demo DB: load schema.sql, seed roles/permissions/nodes, run the permission query (optional but impressive)
- ☐ Practice writing the recursive CTE on paper or a blank editor — you may be asked live
- ☐ Prepare your "AI usage" answer honestly — what you used it for, what decisions you made yourself
- ☐ Review your Thoughtworks Node.js + PostgreSQL experience (Compensation Uploader, Workday integration)
- ☐ Prepare 2–3 questions to ask them (see below)
- ☐ Test your Teams setup — audio, video, screen share — the night before
- ☐ Have your schema.sql open and ready to share your screen



Questions to Ask Them



Asking good questions shows genuine interest and technical depth. Aim for 2–3 at the end.



"Is the tree structure the core domain of your product? How deep do real-world trees get?"

Shows you're thinking about production scale and real use cases.



"How do you currently handle permission caching — computed at query time or cached?"

Shows you've thought about performance beyond the schema design.



"What's the current tech stack for the backend — Node.js, TypeScript, PostgreSQL?"

Practical and helps you understand the role better.



"What's the biggest technical challenge the team is working on right now?"

Great conversation starter that shows you're thinking about contribution.



Strengths to Highlight from Your Resume

Node.js Backend (Guidewire + Thoughtworks)

PostgreSQL + Alembic Migrations

Microservices Architecture

GCP Pub/Sub Event-Driven Systems

40% Release Effort Reduction (Guidewire)

Mentored 6 Junior Engineers

Terraform IaC

Threat Modelling



When answering general technical questions, ground your answers in real experience. E.g., "In the Compensation Uploader project at Thoughtworks, we used Node.js with PostgreSQL and I faced a similar challenge where..." This makes your answers memorable and credible.